DFA Designs for a Java Lexical Analyzer

Programming Languages Course Assignment

Notation: A single circle denotes a state and a double circle an accepting state; the start state is marked by an incoming arrow. Transitions that are not drawn lead to a dead state. *letter*: A–Z, a–z and other Unicode letters; *digit*: 0–9.

Scanning: The DFA whose start transition matches the first character of a lexeme is run until no transition is left (longest match). A character that matches no DFA's start transition is reported as an unrecognized character.

Out of scope: String and character literals, comments.

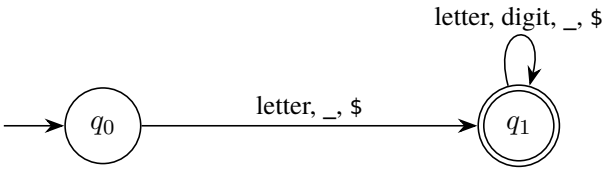

Figure 1. Identifier DFA

Recognizes variable, method and class names. Since a Java identifier cannot start with a digit, digits appear only on the self-loop at q_1. An accepted lexeme is then passed through the DFA in Figure 2 to distinguish it from reserved words.

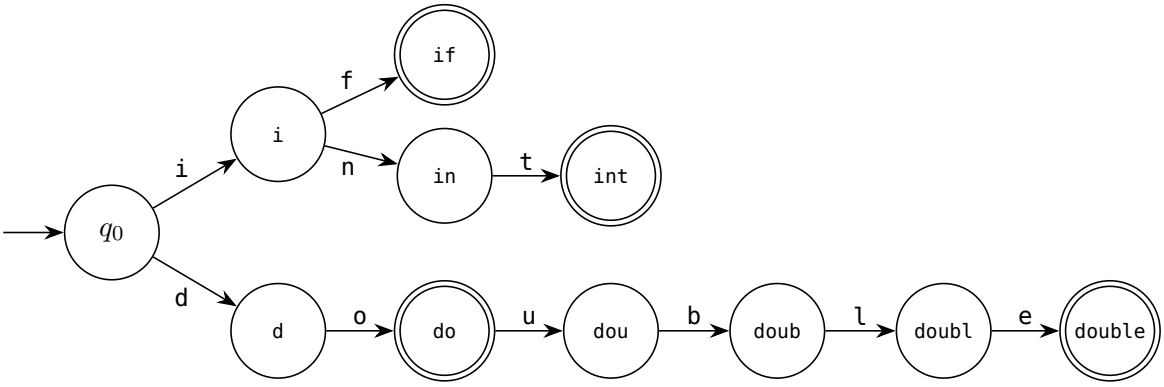

Figure 2. Reserved word (keyword) DFA

A prefix tree built from Java's 51 reserved words (including _); the figure shows the fragment for `if`, `int`, `do` and `double`. If the whole lexeme accepted by the identifier DFA is consumed and the DFA ends in an accepting state, the lexeme is labeled `KEYWORD`; otherwise it is labeled `IDENTIFIER` (e.g. `in`, `integer`). `true`, `false` and `null` are separated in the same way as `BOOLEAN_LITERAL` and `NULL_LITERAL`.

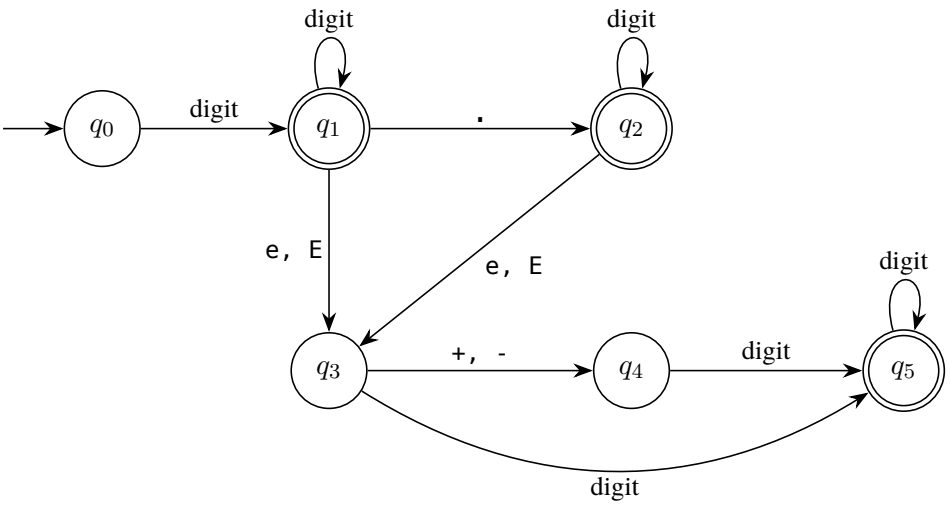

Figure 3. Numeric literal DFA

Recognizes integer and floating-point literals. Accepting states: $q_1 \rightarrow$ `INTEGER_LITERAL`; q_2 and $q_5 \rightarrow$ `FLOAT_LITERAL` (e.g. `42`, `3.14`, `12.`, `2e3`, `1.5e-3`). If the DFA stops in q_3 or q_4 (e.g. `2e+`), or a letter immediately follows the number (e.g. `3abc`), the lexeme is reported as invalid. Hexadecimal and binary notation, type suffixes (`L`, `f`, `d`) and the digit separator (`_`) are out of scope.

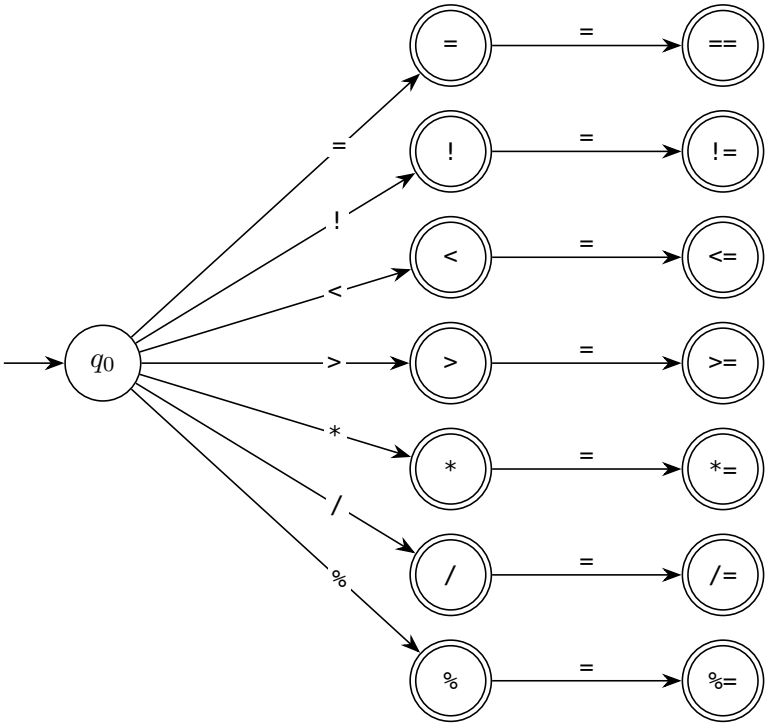

Figure 4. Operator DFA (I)

Recognizes assignment, comparison and arithmetic operators. If `=` follows the first character, a two-character operator (`==`, `!=`, `<=`, `>=`, `*=`, `/=`, `%=`) is read; otherwise the single-character operator is accepted.

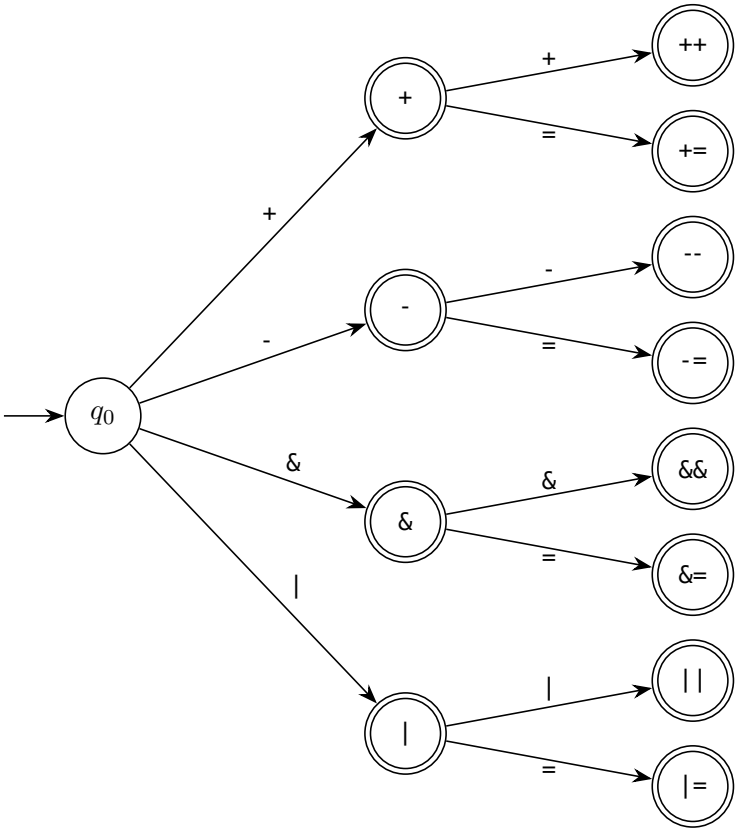

Figure 5. Operator DFA (II)

Recognizes increment/decrement, compound assignment and logical operators. If the first character is repeated, `++`, `--`, `&&` or `||` is read; if `=` follows, `+=`, `-=`, `&=` or `|=` is read. Shift operators (`<<`, `>>`), `^`, `~`, `?`, `:` and `->` are out of scope.

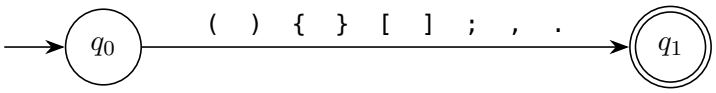

Figure 6. Separator DFA

Recognizes single-character separators; each one produces a separate `SEPARATOR` token. `@`, `...` and `::` are out of scope.

Mert Zeki Öztürk
Student No: 240260115